Week No. 4

Lab Manual: Information Security

Ciphers with OpenSSL

Lab Objectives

- Understand and apply symmetric encryption using OpenSSL commands.
- Perform encryption and decryption using AES, DES, and 3DES algorithms.
- Generate RSA key pairs and use them for encryption and decryption.
- Observe differences between symmetric and asymmetric encryption approaches.

Lab Overview

This lab focuses on hands-on implementation of cryptographic techniques using OpenSSL. Students will create plaintext files, perform symmetric encryption using AES, DES, and 3DES, generate RSA public/private key pairs, encrypt and decrypt messages, and verify whether the recovered plaintext matches the original data.

Experiment 1 – OpenSSL Setup and Plaintext Preparation

OpenSSL is a command-line toolkit used to perform cryptographic operations such as encryption, decryption, random-data generation, key generation, and key inspection. In this experiment, students will verify the OpenSSL installation and prepare a working directory and plaintext file for later experiments.

Steps

Step 1 — Open Ubuntu Terminal

Open the Ubuntu Terminal using the keyboard shortcut:

```
Ctrl + Alt + T
```

Step 2 — Verify OpenSSL Installation

Execute:

```
openssl version
```

Expected output (example):

```
OpenSSL 3.x.x
```

Step 3 — Install OpenSSL (If Required)

If OpenSSL is not installed, execute:

```
sudo apt update
sudo apt install openssl
```

Verify the installation:

```
openssl version
```

Step 4 — Create a Working Directory

```
mkdir OpenSSL_Lab
cd OpenSSL_Lab
```

Step 5 — Create and Verify a Plaintext File

```
echo Hello Students > plaintext.txt
cat plaintext.txt
```

Expected output:

```
Hello Students
```

Step 6 — Generate Random Key Material

Generate a 32-byte (256-bit) random key, a 16-byte (128-bit) IV, and random bytes in hexadecimal format:

```
openssl rand -out key.bin 32
openssl rand -out iv.bin 16
openssl rand -hex 32
```

The hexadecimal command displays 64 hexadecimal characters representing 256 bits. The values will be different each time.

Task 1

1. Create the OpenSSL_Lab directory and plaintext.txt file.
2. Take a screenshot showing the plaintext file content and one random hexadecimal output.



Experiment 2 – AES Encryption and Decryption

Theory

AES (Advanced Encryption Standard) is a modern symmetric encryption algorithm. In symmetric encryption, the same secret is used for encryption and decryption. AES-256 uses a 256-bit key. In this experiment, OpenSSL derives the encryption key from a password using PBKDF2.

Steps

Step 1 — Encrypt File Using AES-256-CBC

```
openssl enc -aes-256-cbc -salt -pbkdf2 -in plaintext.txt -out ciphertext.enc -pass pass:mysecretkey
```

Command explanation:

- openssl enc: performs symmetric encryption or decryption.
- -aes-256-cbc: specifies AES-256-CBC encryption.
- -salt: uses a random salt during password-based key derivation.
- -pbkdf2: uses PBKDF2 for password-based key derivation.
- -in plaintext.txt: specifies the input file.
- -out ciphertext.enc: specifies the encrypted output file.
- -pass pass:mysecretkey: specifies the demonstration password.

Step 2 — Decrypt File Using AES

```
openssl enc -aes-256-cbc -d -pbkdf2 -in ciphertext.enc -out decrypted.txt -pass pass:mysecretkey
```

Step 3 — Verify the Decrypted File

```
cat decrypted.txt
```

Expected output:

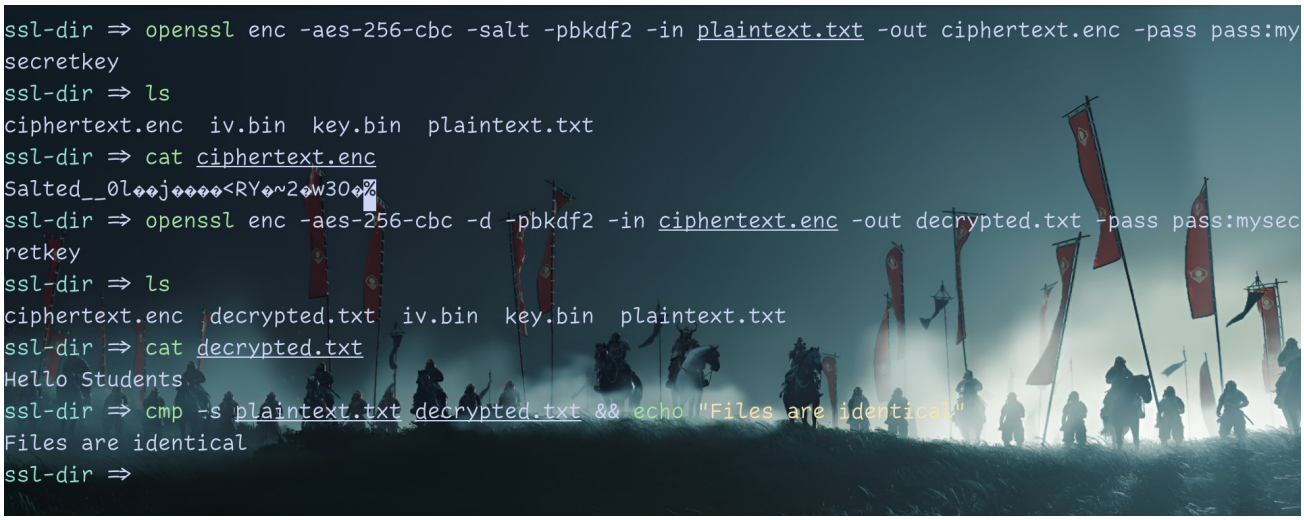
```
Hello Students
```

Step 4 — Compare Original and Decrypted Files

```
cmp -s plaintext.txt decrypted.txt && echo "Files are identical"
```

Expected output:

```
Files are identical
```



Experiment 3 – DES Encryption and Decryption

Theory

DES (Data Encryption Standard) is an older symmetric encryption algorithm with a 56-bit effective key. DES is considered insecure for modern systems and is included here only for educational comparison.

Steps

Step 1 — Encrypt File Using DES

```
openssl enc -provider default -provider legacy -des-cbc -salt -pbkdf2 -in plaintext.txt -out ciphertext_des.enc -pass pass:deskey
```

Step 2 — Decrypt File Using DES

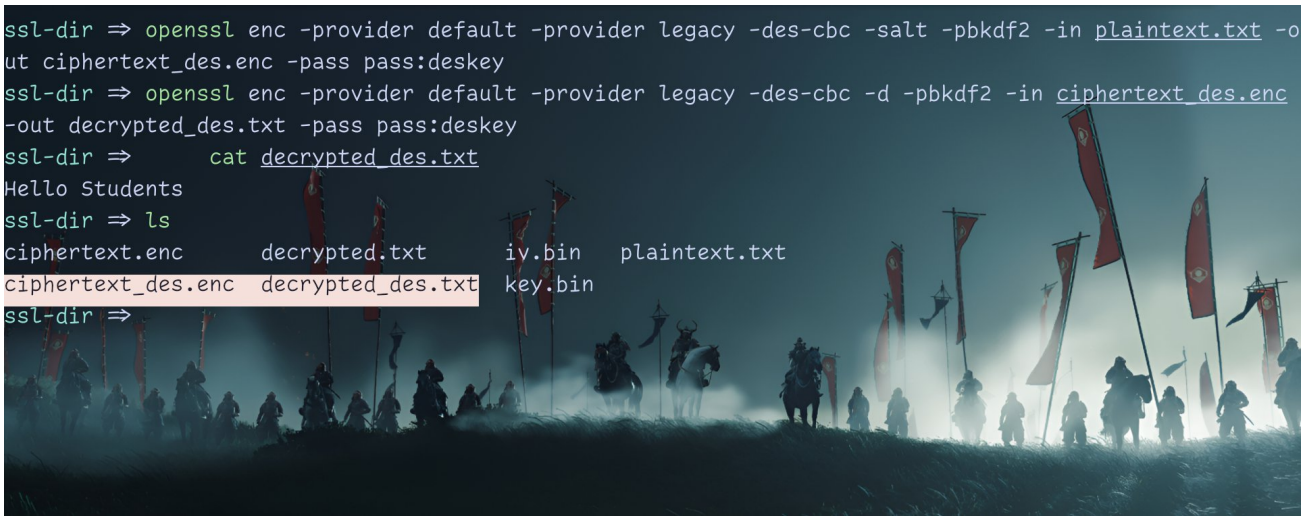
```
openssl enc -provider default -provider legacy -des-cbc -d -pbkdf2 -in ciphertext_des.enc -out decrypted_des.txt -pass pass:deskey
```

Step 3 — Verify the Decrypted File

```
cat decrypted_des.txt
```

Expected output:

```
Hello Students
```



Experiment 4 – Triple DES (3DES) Encryption and Decryption

Triple DES (3DES) extends DES by applying DES operations three times. It provides stronger protection than single DES, but it is slower than AES and is now considered outdated for new systems.

Steps

Step 1 — Encrypt File Using 3DES

```
openssl enc -des-ede3-cbc -salt -pbkdf2 -in plaintext.txt -out ciphertext_3des.enc -pass pass:tripledes
```

Step 2 — Decrypt File Using 3DES

```
openssl enc -des-ede3-cbc -d -pbkdf2 -in ciphertext_3des.enc -out decrypted_3des.txt -pass pass:tripledes
```

Step 3 — Verify the Decrypted File

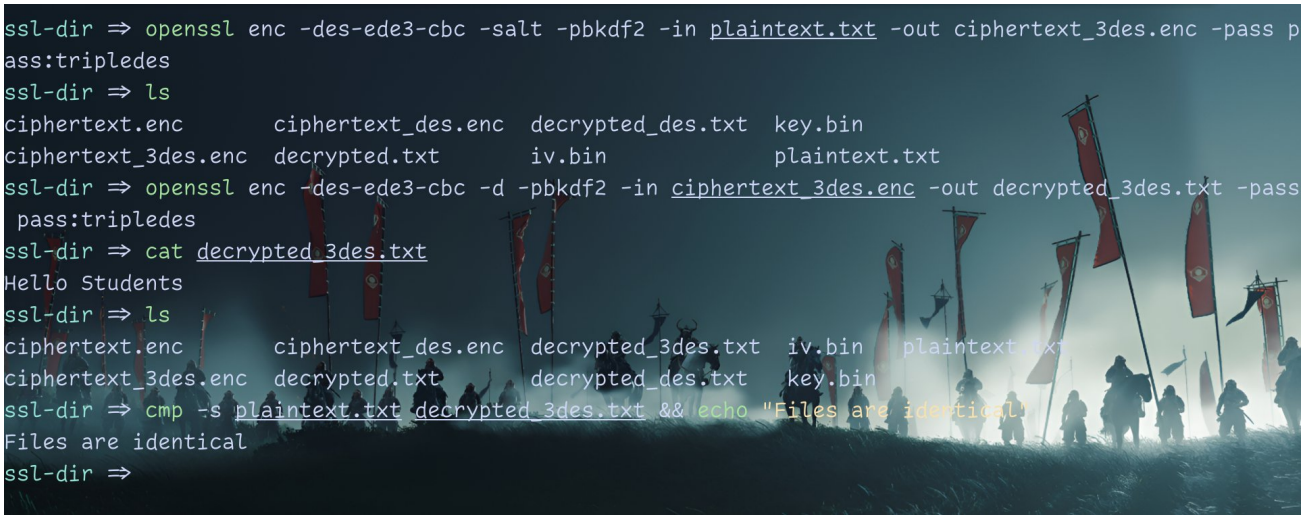
```
cat decrypted_3des.txt
```

Expected output:

```
Hello Students
```

Task 4

1. Perform 3DES encryption and decryption.
2. Compare the 3DES decrypted file with the original plaintext using cmp.



Experiment 5 – RSA Key Generation, Encryption and Decryption

RSA is an asymmetric encryption algorithm that uses two mathematically related keys. The public key can be shared and is used for encryption, while the private key must remain confidential and is used for decryption. RSA is normally used for small data, keys, or hybrid cryptosystems rather than direct encryption of large files.

Steps

Step 1 — Generate RSA Private Key

```
openssl genpkey -algorithm RSA -out private.pem -pkeyopt rsa_keygen_bits:2048
```

Expected result: private.pem will be created.

Step 2 — Generate RSA Public Key

```
openssl pkey -in private.pem -pubout -out public.pem
```

Expected result: public.pem will be created.

Step 3 — Verify Key Files

```
ls -l *.pem
```

You should see:

- private.pem
- public.pem

Step 4 — Create a Message File

```
echo Information Security Lab > message.txt
cat message.txt
```

Expected output:

```
Information Security Lab
```

Step 5 — Encrypt Using RSA Public Key

```
openssl pkeyutl -encrypt -pubin -inkey public.pem -in message.txt -out message.enc -pkeyopt
rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256
```

The command uses RSA-OAEP padding with SHA-256.

Step 6 — Decrypt Using RSA Private Key

```
openssl pkeyutl -decrypt -inkey private.pem -in message.enc -out message_decrypted.txt -pkeyopt
rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256
```

Step 7 — Verify Decrypted Message

```
cat message_decrypted.txt
```

Expected output:

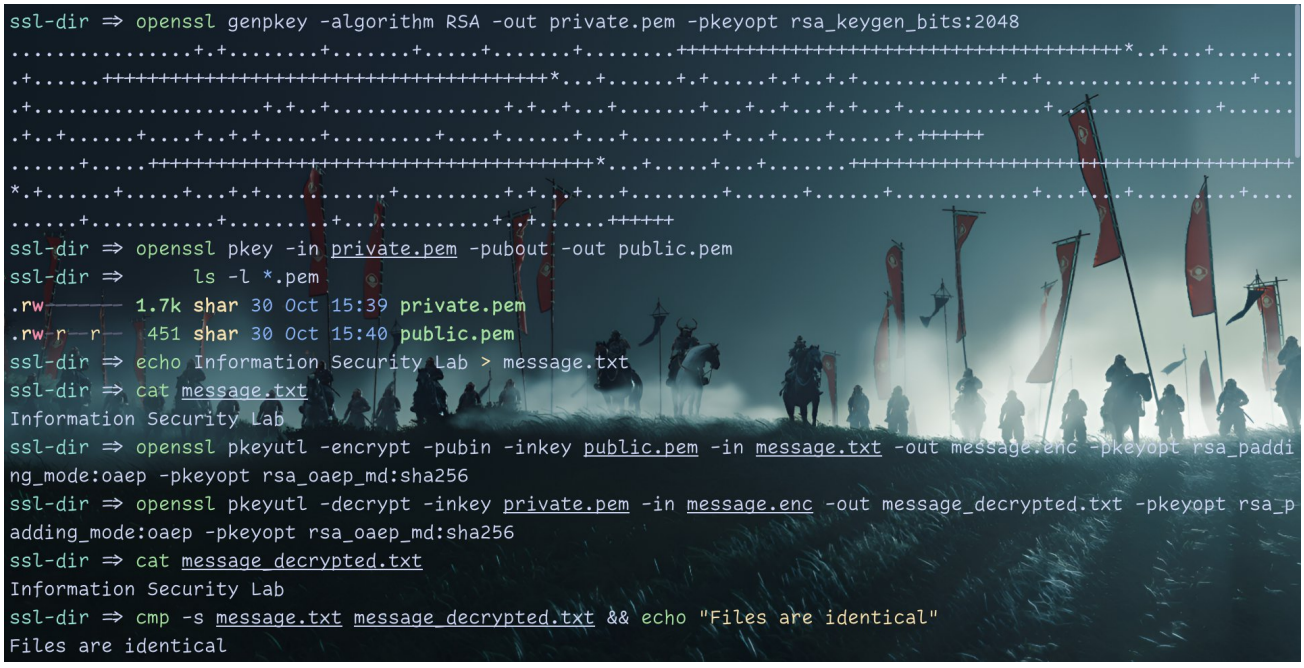
```
Information Security Lab
```

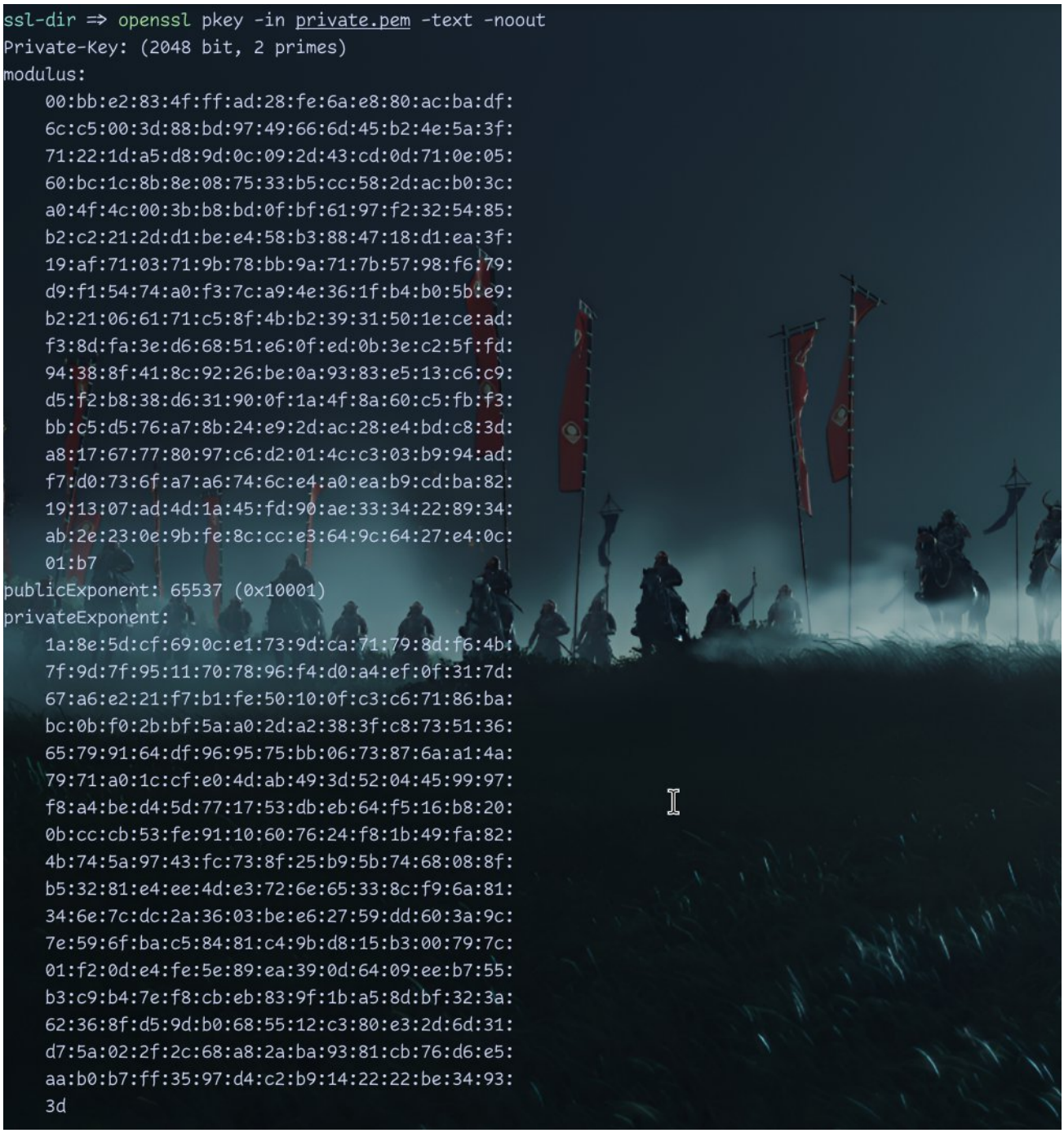
Step 8 — View RSA Key Information

```
openssl pkey -in private.pem -text -noout
```

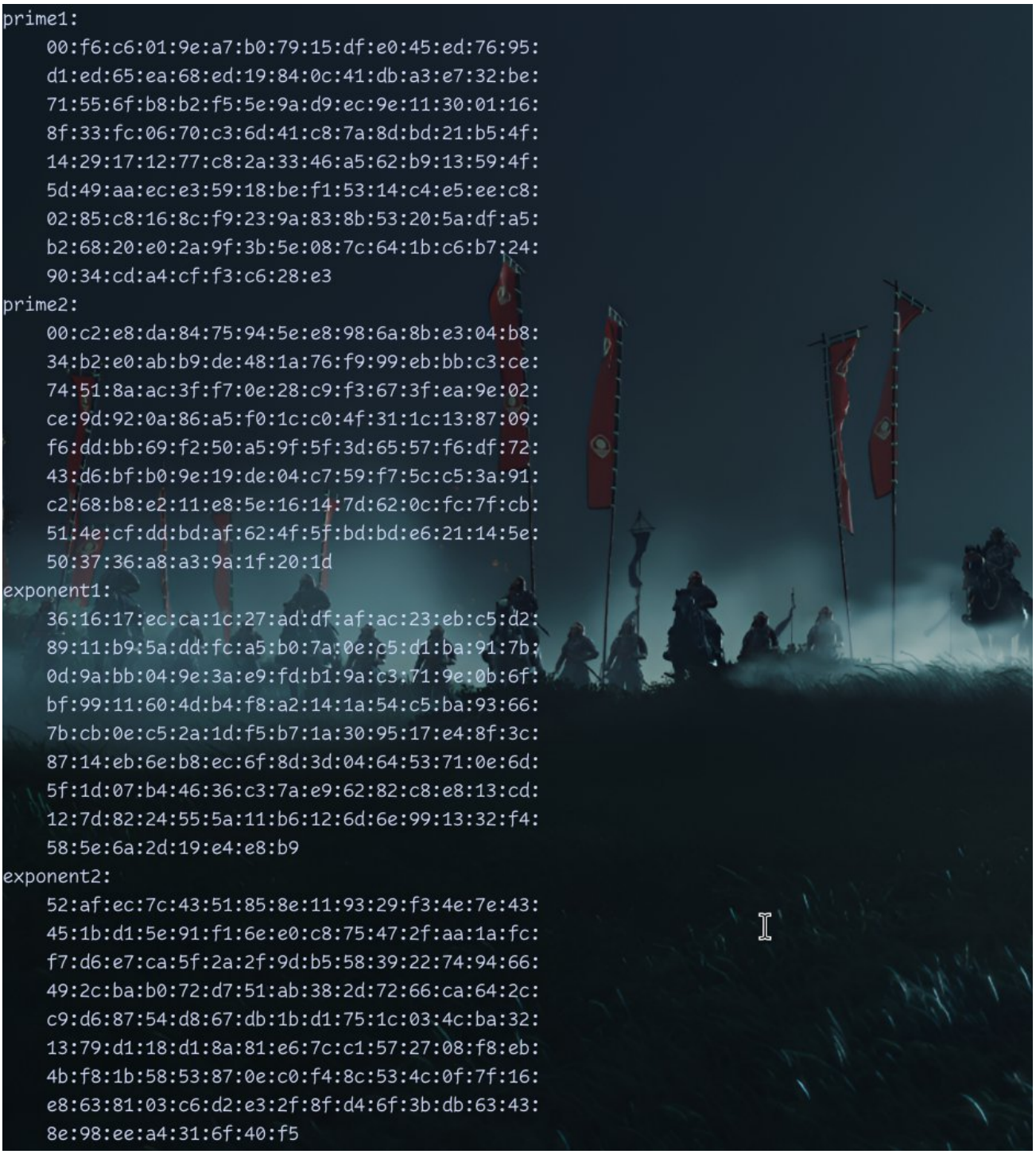
Task 5

1. Generate a 2048-bit RSA private key and its public key.
2. Encrypt message.txt with the public key and decrypt it with the private key.
3. Attach screenshots showing the generated key files and the recovered plaintext.

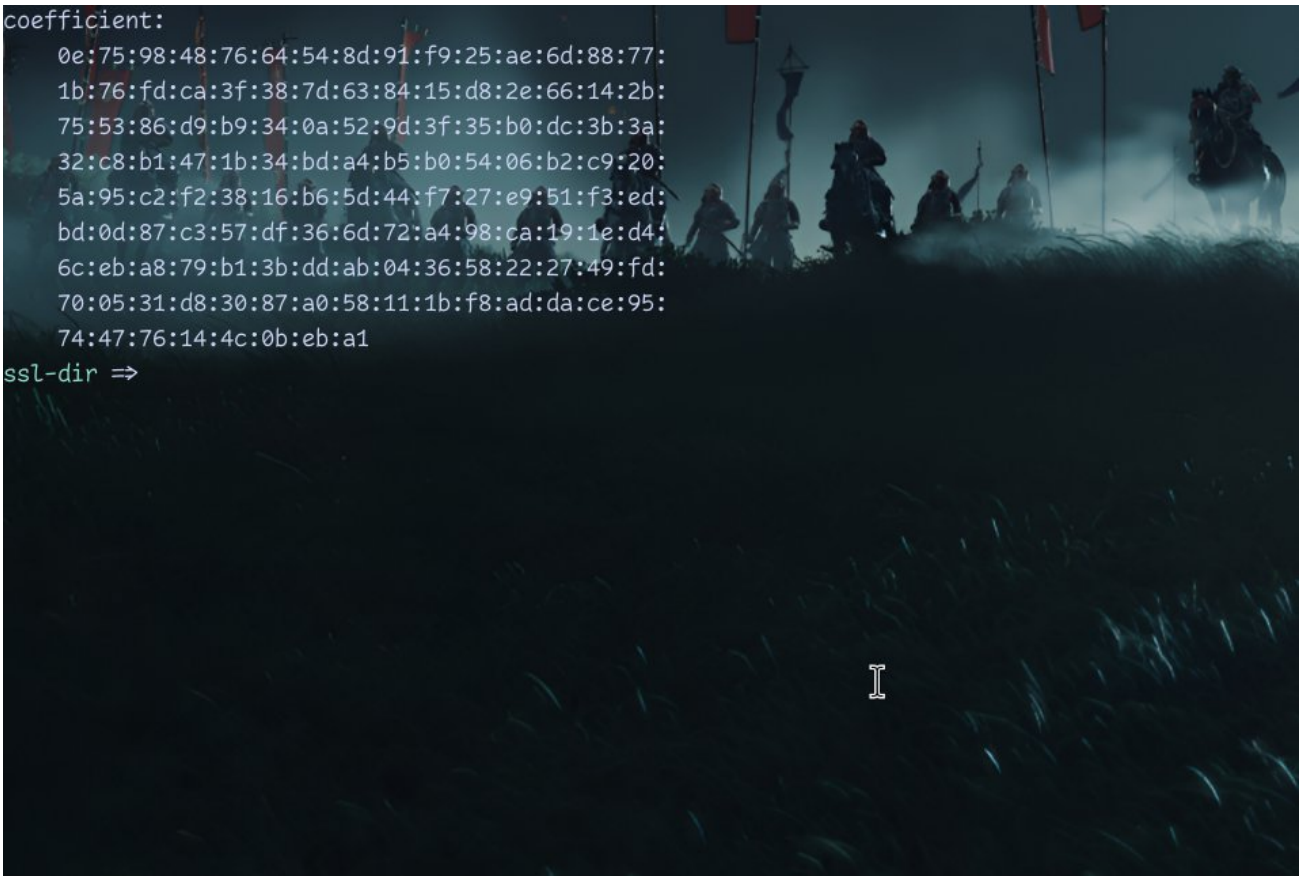




```
ssl-dir => openssl pkey -in private.pem -text -noout
Private-Key: (2048 bit, 2 primes)
modulus:
    00:bb:e2:83:4f:ff:ad:28:fe:6a:e8:80:ac:ba:df:
    6c:c5:00:3d:88:bd:97:49:66:6d:45:b2:4e:5a:3f:
    71:22:1d:a5:d8:9d:0c:09:2d:43:cd:0d:71:0e:05:
    60:bc:1c:8b:8e:08:75:33:b5:cc:58:2d:ac:b0:3c:
    a0:4f:4c:00:3b:b8:bd:0f:bf:61:97:f2:32:54:85:
    b2:c2:21:2d:d1:be:e4:58:b3:88:47:18:d1:ea:3f:
    19:af:71:03:71:9b:78:bb:9a:71:7b:57:98:f6:79:
    d9:f1:54:74:a0:f3:7c:a9:4e:36:1f:b4:b0:5b:e9:
    b2:21:06:61:71:c5:8f:4b:b2:39:31:50:1e:ce:ad:
    f3:8d:fa:3e:d6:68:51:e6:0f:ed:0b:3e:c2:5f:fd:
    94:38:8f:41:8c:92:26:be:0a:93:83:e5:13:c6:c9:
    d5:f2:b8:38:d6:31:90:0f:1a:4f:8a:60:c5:fb:f3:
    bb:c5:d5:76:a7:8b:24:e9:2d:ac:28:e4:bd:c8:3d:
    a8:17:67:77:80:97:c6:d2:01:4c:c3:03:b9:94:ad:
    f7:d0:73:6f:a7:a6:74:6c:e4:a0:ea:b9:cd:ba:82:
    19:13:07:ad:4d:1a:45:fd:90:ae:33:34:22:89:34:
    ab:2e:23:0e:9b:fe:8c:cc:e3:64:9c:64:27:e4:0c:
    01:b7
publicExponent: 65537 (0x10001)
privateExponent:
    1a:8e:5d:cf:69:0c:e1:73:9d:ca:71:79:8d:f6:4b:
    7f:9d:7f:95:11:70:78:96:f4:d0:a4:ef:0f:31:7d:
    67:a6:e2:21:f7:b1:fe:50:10:0f:c3:c6:71:86:ba:
    bc:0b:f0:2b:bf:5a:a0:2d:a2:38:3f:c8:73:51:36:
    65:79:91:64:df:96:95:75:bb:06:73:87:6a:a1:4a:
    79:71:a0:1c:cf:e0:4d:ab:49:3d:52:04:45:99:97:
    f8:a4:be:d4:5d:77:17:53:db:eb:64:f5:16:b8:20:
    0b:cc:cb:53:fe:91:10:60:76:24:f8:1b:49:fa:82:
    4b:74:5a:97:43:fc:73:8f:25:b9:5b:74:68:08:8f:
    b5:32:81:e4:ee:4d:e3:72:6e:65:33:8c:f9:6a:81:
    34:6e:7c:dc:2a:36:03:be:e6:27:59:dd:60:3a:9c:
    7e:59:6f:ba:c5:84:81:c4:9b:d8:15:b3:00:79:7c:
    01:f2:0d:e4:fe:5e:89:ea:39:0d:64:09:ee:b7:55:
    b3:c9:b4:7e:f8:cb:eb:83:9f:1b:a5:8d:bf:32:3a:
    62:36:8f:d5:9d:b0:68:55:12:c3:80:e3:2d:6d:31:
    d7:5a:02:2f:2c:68:a8:2a:ba:93:81:cb:76:d6:e5:
    aa:b0:b7:ff:35:97:d4:c2:b9:14:22:22:be:34:93:
    3d
```



```
prime1:
    00:f6:c6:01:9e:a7:b0:79:15:df:e0:45:ed:76:95:
    d1:ed:65:ea:68:ed:19:84:0c:41:db:a3:e7:32:be:
    71:55:6f:b8:b2:f5:5e:9a:d9:ec:9e:11:30:01:16:
    8f:33:fc:06:70:c3:6d:41:c8:7a:8d:bd:21:b5:4f:
    14:29:17:12:77:c8:2a:33:46:a5:62:b9:13:59:4f:
    5d:49:aa:ec:e3:59:18:be:f1:53:14:c4:e5:ee:c8:
    02:85:c8:16:8c:f9:23:9a:83:8b:53:20:5a:df:a5:
    b2:68:20:e0:2a:9f:3b:5e:08:7c:64:1b:c6:b7:24:
    90:34:cd:a4:cf:f3:c6:28:e3
prime2:
    00:c2:e8:da:84:75:94:5e:e8:98:6a:8b:e3:04:b8:
    34:b2:e0:ab:b9:de:48:1a:76:f9:99:eb:bb:c3:ce:
    74:51:8a:ac:3f:f7:0e:28:c9:f3:67:3f:ea:9e:02:
    ce:9d:92:0a:86:a5:f0:1c:c0:4f:31:1c:13:87:09:
    f6:dd:bb:69:f2:50:a5:9f:5f:3d:65:57:f6:df:72:
    43:d6:bf:b0:9e:19:de:04:c7:59:f7:5c:c5:3a:91:
    c2:68:b8:e2:11:e8:5e:16:14:7d:62:0c:fc:7f:cb:
    51:4e:cf:dd:bd:af:62:4f:5f:bd:bd:e6:21:14:5e:
    50:37:36:a8:a3:9a:1f:20:1d
exponent1:
    36:16:17:ec:ca:1c:27:ad:df:af:ac:23:eb:c5:d2:
    89:11:b9:5a:dd:fc:a5:b0:7a:0e:c5:d1:ba:91:7b:
    0d:9a:bb:04:9e:3a:e9:fd:b1:9a:c3:71:9e:0b:6f:
    bf:99:11:60:4d:b4:f8:a2:14:1a:54:c5:ba:93:66:
    7b:cb:0e:c5:2a:1d:f5:b7:1a:30:95:17:e4:8f:3c:
    87:14:eb:6e:b8:ec:6f:8d:3d:04:64:53:71:0e:6d:
    5f:1d:07:b4:46:36:c3:7a:e9:62:82:c8:e8:13:cd:
    12:7d:82:24:55:5a:11:b6:12:6d:6e:99:13:32:f4:
    58:5e:6a:2d:19:e4:e8:b9
exponent2:
    52:af:ec:7c:43:51:85:8e:11:93:29:f3:4e:7e:43:
    45:1b:d1:5e:91:f1:6e:e0:c8:75:47:2f:aa:1a:fc:
    f7:d6:e7:ca:5f:2a:2f:9d:b5:58:39:22:74:94:66:
    49:2c:ba:b0:72:d7:51:ab:38:2d:72:66:ca:64:2c:
    c9:d6:87:54:d8:67:db:1b:d1:75:1c:03:4c:ba:32:
    13:79:d1:18:d1:8a:81:e6:7c:c1:57:27:08:f8:eb:
    4b:f8:1b:58:53:87:0e:c0:f4:8c:53:4c:0f:7f:16:
    e8:63:81:03:c6:d2:e3:2f:8f:d4:6f:3b:db:63:43:
    8e:98:ee:a4:31:6f:40:f5
```



Experiment 6 – Verification and Ciphertext Comparison

Ciphertext generated by different algorithms may differ in size and structure because each algorithm uses different key sizes, block sizes, padding methods, and metadata. Verification confirms whether decryption restored the exact original plaintext.

Steps

Step 1 — Display Encrypted Files

```
ls -l *.enc
```

Observe the file sizes of ciphertext.enc, ciphertext_des.enc, ciphertext_3des.enc, and message.enc.

Step 2 — Identify Encrypted File Type

```
file ciphertext.enc
```

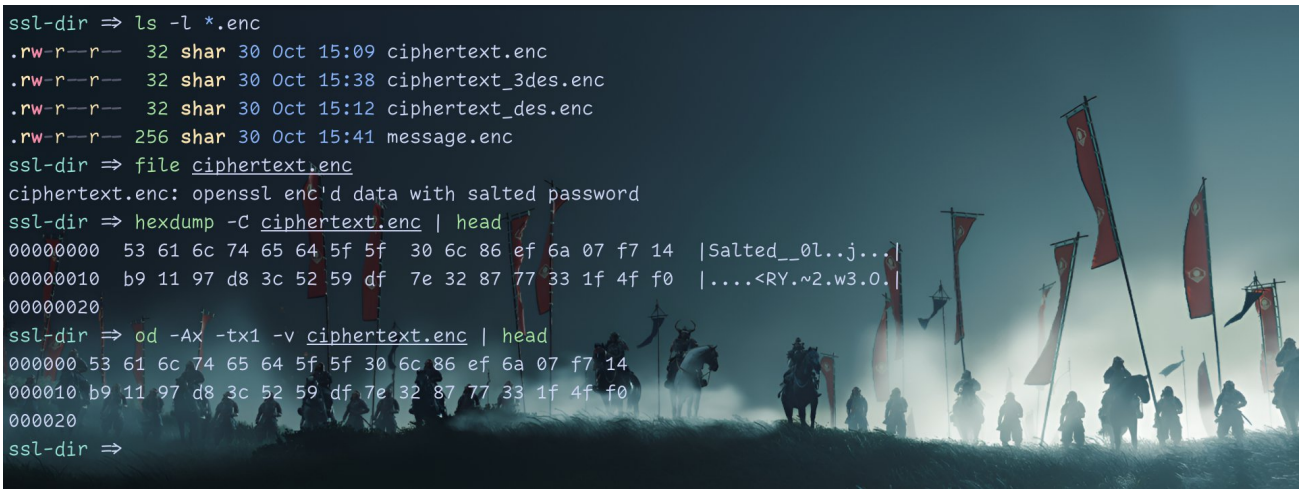
For password-based OpenSSL encryption with a salt, the file may be identified as OpenSSL encrypted data.

Step 3 — Display Encrypted Data in Hexadecimal Format

```
hexdump -C ciphertext.enc | head
```

If hexdump is unavailable, use:

```
od -Ax -tx1 -v ciphertext.enc | head
```



Step 4 — Compare Original and Decrypted Files

```
cmp plaintext.txt decrypted.txt
cmp plaintext.txt decrypted_des.txt
cmp plaintext.txt decrypted_3des.txt
cmp message.txt message_decrypted.txt
```

If the compared files are identical, cmp normally produces no output.



Lab Questions:

1. **Differentiate symmetric and asymmetric encryption in terms of keys, speed, and typical use.**

Feature	Symmetric Encryption	Asymmetric Encryption
Keys	One shared key	Public and private keys
Speed	Faster	Slower
Use	Encrypting large amounts of data	Secure key exchange and digital signatures
Example	AES, DES	RSA

2. **Explain why RSA is not suitable for encrypting large files directly.**

RSA is computationally slow and can only encrypt data smaller than its key size allows. Therefore, it is inefficient for large files.

3. **Explain how symmetric and asymmetric encryption can be combined in a hybrid cryptosystem.**

A hybrid cryptosystem combines both encryption methods. **Asymmetric encryption** securely exchanges a randomly generated symmetric key, while **symmetric encryption** uses that key to encrypt and decrypt the actual data. This provides both security and speed.

4. **Compare AES, DES, and 3DES and identify which one should be preferred for modern applications.**

Algorithm	Key size	Security	Speed
DES	56-bit	Weak & outdated	Fast
3DES	112 or 168-bit nominal	Outdated	Slow
AES	128, 192, or 256-bit	Strong	Fast